CECS 229 – Assignment 8

Matrices

The file, 'pa8.py', contains an incomplete class Matrix. The init method initializes a 2D list called rows. Please note that all Matrix index locations, such as i and j, are 1 based rather than 0 based, so they will always need to be offset before accessing the 2D list of elements (ie. location i=1, j=1 is the upper left corner at index [0][0]). Complete the code for the following methods:

1. `set_row(self, i, v)` – this method assigns the vector v to the specified row of the matrix. The number of elements in v should match the number of columns of the matrix.
    a. i – the index of the row to be overwritten with vector v (1 based).
    b. v – the vector to be placed at row i.
    c. Raise a ValueError exception if the length of v is invalid.
2. `set_col(self, j, u)` – this method assigns the vector u to the specified column of the matrix. The number of elements in u should match the number of rows of the matrix.
    a. j – the index of the column to be overwritten with vector u (1 based).
    b. u – the vector to be placed at column j.
    c. Raise a ValueError exception if the length of u is invalid.
3. `set_entry(self, i, j, a)` – this method assigns the element a to the specified row and column of the matrix.
    a. i – the index of the row (1 based).
    b. j – the index of the column (1 based).
    c. a – the element to be placed at location i, j.
4. `get_row(self, i)` – this method returns the vector at the specified row of the matrix.
    a. i – the index of the row to be accessed (1 based).
    b. Returns the vector at row i.
5. `get_col(self, j)` – this method returns the vector at the specified column of the matrix.
    a. j – the index of the column to be accessed (1 based).
    b. Returns the vector at column j.
6. `get_element(self, i, j)` – this method returns the element at the specified row and column of the matrix.
    a. i – the index of the row to be accessed (1 based).
    b. j – the index of the column to be accessed (1 based).
    c. Returns the element at row i and column j.
7. `get_columns(self)` – this method returns the matrix as a list of column vectors.
    a. Returns a list of column vectors.
8. `get_rows(self)` – this method returns the matrix as a list of row vectors.
    a. Returns a list of row vectors.
9. `dim(self)` – this method returns the dimensions of the matrix as a tuple: (rows, cols).
    a. Returns a tuple with the number of rows and columns.
10. `get_diag(self)` – this method returns the elements that are in the diagonal starting from location i = 1, j = 1.
    a. Returns a list of elements that are in the diagonal.

11. `__add__(self, other)` – overloads the '+' operator for matrices.
    a. self – the left operand, the first matrix to be added.
    b. other – the right operand, the second matrix to be added.
    c. Return a new matrix that is the sum of the left and right operands.
    d. Raise a TypeError exception if the other parameter is not a matrix.
    e. Raise a ValueError exception if the dimensions of the two matrices do not match.
12. `__sub__(self, other)` – overloads the '-' operator for matrices.
    a. self – the left operand, the first matrix to be subtracted.
    b. other – the right operand, the second matrix to be subtracted.
    c. Return a new matrix that is the result of subtracting the right operand from the left.
    d. Raise a TypeError exception if the other parameter is not a matrix.
    e. Raise a ValueError exception if the dimensions of the two matrices do not match.
13. `__mul__(self, other)` – overloads the '*' operator for multiplying a matrix with another matrix or a matrix with a scalar (ie. an int or a float).
    a. self – the left operand, the matrix to be multiplied.
    b. other – the right operand, this will either be a scalar or another matrix.
    c. If other is a scalar, then return a new matrix where each of the elements is a result of multiplying the element from the matrix by the scalar. Note, the dimensions of this result should be the same as the dimensions of the original matrix.
    d. If other is a matrix, then return a new matrix where each element is the result of doing matrix multiplication (ie. the dot product of each row and column of the two matrices). Note, the dimensions of this result should be the number of rows of the first matrix by the number of columns in the second matrix).
    e. Raise a TypeError exception if the other parameter is not a matrix or scalar.
    f. Raise a ValueError if 'other' was a matrix and it does not have the correct number of rows.
14. `__rmul__(self, other)` – overloads the reverse '*' operator for multiplying a scalar with a vector.
    a. self – the right operand, the matrix to be multiplied.
    b. other – the left operand, the scalar to be multiplied.
    c. The returned value will be a new matrix, resulting from doing scalar multiplication. Note, the dimensions of this result should be the same as the dimensions of the original matrix.
    d. Raise a TypeError exception if the other parameter is not a scalar.
15. `__eq__(self, other)` – overloads the '==' operator for comparing two matrices.
    a. self – the left operand, the matrix to be compared.
    b. other – the right operand, the other matrix to be compared.
    c. Returns true if all elements are the same and in the same locations.

Notes:

1. Please place your name in the comment block at the top of the provided file.
2. Fill in the necessary code to complete the functions. Write your code where specified (ie. the TODOs). Do not modify the init or str methods. Do not add additional attributes or methods. Do not modify the file name, function names, or parameters otherwise the

program and tests will not work. Return the results in the format specified (ie. correct types and container types), otherwise the program and tests will not work properly.

3. Your solutions may not use any libraries or functions that perform the equivalent task.
4. Test your code using the provided 'main.py' file. You can write additional tests in the 'your_tester.py' file to further check your solutions. Note: just because the main tests pass does not guarantee that it will pass the tests on Gradescope.
5. When you are finished, submit your 'pa8.py' file to Gradescope on Canvas for further testing and grading. It should allow you to make repeated submissions.